

Caio Collino Troiano
Thiago Pereira Lins da Silva
Vinicius Vianna Egydio de Carvalho

Computação Paralela - Sistemas Distribuídos entre VMs

São Paulo
2026

Sumário

1	INTRODUÇÃO	2
1.1	Problema Escolhido	2
1.2	Configuração do Ambiente	2
1.2.1	Máquinas Virtuais	2
1.2.2	Conectividade entre as Máquinas	3
2	ESTRATÉGIA DE PARALELIZAÇÃO	4
2.1	Divisão de Carga	4
2.2	Comunicação entre Processos	4
3	CÓDIGO-FONTE	5
4	COMPILAÇÃO E EXECUÇÃO	9
4.1	Compilação	9
4.2	Execução	9
5	RESULTADOS OBTIDOS	10
6	DIFICULDADES ENCONTRADAS E APRENDIZADOS	11

1 Introdução

Este relatório apresenta a implementação e execução de uma aplicação distribuída utilizando a biblioteca MPI (Message Passing Interface) em um cluster composto por máquinas virtuais configuradas no VirtualBox. O objetivo da atividade foi aplicar conceitos de computação distribuída para resolver um problema computacionalmente intensivo por meio da distribuição de carga entre múltiplos processos.

1.1 Problema Escolhido

Título: Geração do Conjunto de Mandelbrot utilizando processamento paralelo com MPI

Edição: 12ª maratona de programação paralela (MACKENZIE)

O problema consiste em gerar uma representação ASCII do conjunto de Mandelbrot, um conhecido fractal matemático obtido por meio da iteração da equação complexa:

$$z_{n+1} = z_n^2 + c$$

onde z e c são números complexos.

A geração do fractal exige a execução de um grande número de cálculos independentes para cada ponto da imagem, tornando-o um problema adequado para paralelização.

1.2 Configuração do Ambiente

1.2.1 Máquinas Virtuais

As máquinas virtuais foram configuradas utilizando o VirtualBox, contendo sistema operacional Linux (Alpine) e suporte à biblioteca OpenMPI.

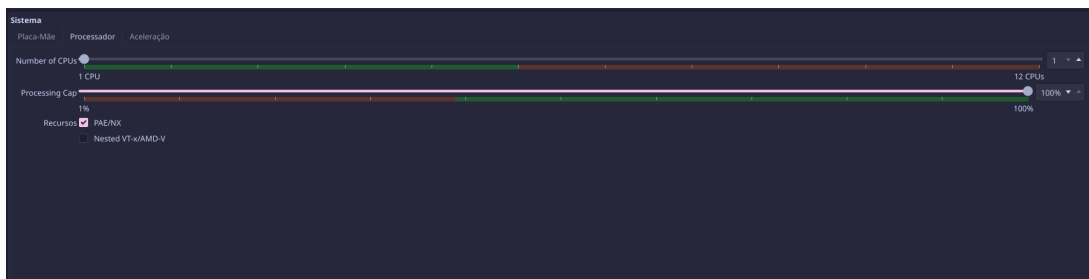


Figura 1 – Configuração das máquinas virtuais pt. 1

2 Estratégia de Paralelização

2.1 Divisão de Carga

A aplicação distribui as linhas da imagem do conjunto de Mandelbrot entre todos os processos MPI disponíveis.

Inicialmente, o processo mestre (rank 0) recebe os parâmetros de entrada:

- Número de linhas;
- Número de colunas;
- Número máximo de iterações.

Esses valores são enviados para todos os processos por meio da função:

`MPI_Bcast()`

A quantidade de linhas atribuídas a cada processo é calculada através da divisão inteira:

```
base_rows = max_row / size
```

Caso a divisão não seja exata, as linhas restantes são distribuídas entre os primeiros processos.

Cada processo calcula apenas o subconjunto de linhas sob sua responsabilidade, reduzindo significativamente o tempo total de processamento.

2.2 Comunicação entre Processos

A comunicação ocorre em duas etapas principais:

1. Distribuição dos parâmetros de execução utilizando `MPI_Bcast`.
2. Coleta dos resultados parciais utilizando `MPI_Gatherv`.

A função `MPI_Gatherv` foi utilizada porque diferentes processos podem receber quantidades distintas de linhas quando o número total de linhas não é divisível pela quantidade de processos.

O processo mestre reúne todos os blocos calculados e monta a imagem final do fractal.

3 Código-Fonte

O código-fonte completo da aplicação é apresentado a seguir.

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int max_row, max_column, max_n;

    if (rank == 0) {
        if (argc != 4) {
            fprintf(stderr, "Uso: %s linhas colunas iteracoes\n", argv[0]);
            MPI_Abort(MPI_COMM_WORLD, 1);
        }

        max_row = atoi(argv[1]);
        max_column = atoi(argv[2]);
        max_n = atoi(argv[3]);
    }

    MPI_Bcast(&max_row, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&max_column, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&max_n, 1, MPI_INT, 0, MPI_COMM_WORLD);

    int base_rows = max_row / size;
    int remainder = max_row % size;

    int local_rows = base_rows + (rank < remainder ? 1 : 0);

    int start_row = rank * base_rows +
        (rank < remainder ? rank : remainder);
```

```

char *local = malloc(local_rows * max_column);

for (int r = 0; r < local_rows; r++) {
    int global_r = start_row + r;

    for (int c = 0; c < max_column; c++) {
        float zr = 0.0f;
        float zi = 0.0f;

        float cr =
            (float)c * 2.0f / max_column - 1.5f;

        float ci =
            (float)global_r * 2.0f / max_row - 1.0f;

        int n = 0;

        while ((zr * zr + zi * zi) < 4.0f &&
            ++n < max_n) {

            float new_zr =
                zr * zr - zi * zi + cr;

            float new_zi =
                2.0f * zr * zi + ci;

            zr = new_zr;
            zi = new_zi;
        }

        local[r * max_column + c] =
            (n == max_n) ? '#' : '.';
    }
}

char *global = NULL;
int *recvcunts = NULL;
int *displs = NULL;

if (rank == 0) {
    global = malloc(max_row * max_column);

```

```

recvcounts = malloc(size * sizeof(int));
displs = malloc(size * sizeof(int));

int offset = 0;

for (int p = 0; p < size; p++) {
    int rows =
        base_rows + (p < remainder ? 1 : 0);

    recvcounts[p] = rows * max_column;
    displs[p] = offset;

    offset += recvcounts[p];
}
}

MPI_Gatherv(
    local,
    local_rows * max_column,
    MPI_CHAR,
    global,
    recvcounts,
    displs,
    MPI_CHAR,
    0,
    MPI_COMM_WORLD
);

if (rank == 0) {

    for (int r = 0; r < max_row; r++) {
        for (int c = 0; c < max_column; c++) {
            putchar(global[r * max_column + c]);
        }
        putchar('\n');
    }

    free(global);
    free(recvcounts);
    free(displs);
}

```



```
    free(local);

    MPI_Finalize();

    return 0;
}
```

4 Compilação e Execução

4.1 Compilação

A aplicação foi compilada utilizando o compilador MPI:

```
mpicc mandelbrot.c -o mandelbrot
```

4.2 Execução

O seguinte comando foi utilizado para executar a aplicação no cluster:

```
mpirun -np 1 --hostfile hosts ./build/mandelbrot 40 80 128000
```

Onde:

- `-np 1`: executa 1 processo MPI;
- `--hostfile hosts`: máquinas participantes;
- `40`: exemplo de número de linhas;
- `80`: exemplo de número de colunas;
- `12800`: número máximo de iterações.

5 Resultados Obtidos

A Figura abaixo apresenta a execução correta da aplicação distribuída e o resultado gerado para a entrada de teste.



Figura 5 – Resultado da execução da aplicação MPI

6 Dificuldades Encontradas e Aprendizados

Durante o desenvolvimento da atividade foi encontrado dificuldades relacionadas à compreensão o funcionamento das operações coletivas do MPI, especialmente a utilização da função `MPI_Gatherv`, necessária para reunir blocos de tamanhos diferentes calculados pelos processos.

Como aprendizado, a atividade permitiu compreender conceitos fundamentais de computação paralela, incluindo:

- Modelo SPMD (Single Program Multiple Data);
- Comunicação coletiva com MPI;
- Balanceamento de carga;
- Distribuição de tarefas entre múltiplos processos;
- Execução distribuída em clusters.

Além disso, foi possível observar na prática como problemas independentes, como o cálculo do conjunto de Mandelbrot, podem ser paralelizados de forma eficiente para reduzir o tempo de processamento.